

19. Soutěžení procesů (o prostředky), problémy s konkurencí, vzájemné vylučování, kritická sekce. předpoklady pro řešení KS, požadované vlastnosti řešení KS, příklady nevhodných SW řešení.

Konkurence procesů:

- Sdílení prostředků
 - CPU, paměť, soubory
- Komunikace procesů
 - přístup ke sdílené paměti, fronty zpráv
- Synchronizace procesů
 - semaforey

Soutěžení procesů (o prostředky)

- OS provádí procesy (pseudo) paralelně
 - proto výsledek výpočtu procesu nesmí záviset na rychlosti jeho provádění
- provádění procesu může ovlivnit chování ostatních procesů v systému
 - některé prostředky nelze (v jednom okamžiku) sdílet
 - přidělení prostředku jednomu procesu omezí ostatní procesy, které požadují tento prostředek
- blokový proces může bez omezení čekat

Provádění jednoho procesu může ovlivnit chování soupeřících procesů; pokud přístup k jednomu prostředku požaduje více procesů současně, lze prostředek přidělit jen jednomu z nich; nelze vždy zaručit, že blokový proces získá přístup k prostředku v budoucnosti.

Problémy s konkurencí

- porušení konzistence dat
 - procesy v tu samou chvíli přistupují ke sdílenému prostředku
- smrtící objekt – deadlock
 - procesy vzájemně čekají na uvolnění prostředků
 - př.: proces 1 používá zařízení A a proces 2 používá zařízení B a jeden chce zařízení toho druhého
- vyhladovění – starvation
 - nekončící čekání procesu na získání prostředku
 - př.: proces s nízkou prioritou čeká na přidělení CPU

Vzájemné vylučování – mutual exclusion (mutex)

V každém okamžiku smí mít přístup ke sdílenému prostředku pouze jeden proces. Musí být zaručena atomicita operace (tzn. operaci nelze přerušit). Tím je zaručena konzistence dat.

Kritická sekce (KS)

- část kódu, kde proces manipuluje se sdíleným prostředkem
- provádění tohoto kódu musí být vzájemně vylučné
 - v každém okamžiku smí být v kritické sekci (pro daný prostředek) pouze jediný proces
 - proces musí žádat o povolení vstupu do kritické sekce

Struktura programu s KS:

- vstupní sekce (entry section)
 - implementace povolení vstupu do KS
- kritická sekce (critical section)
 - manipulace se sdílenými prostředky
- výstupní sekce (exit section)
 - implementace uvolnění KS pro jiné procesy
- zbytková sekce (remainder section)
 - zbytek kódu procesu

Řešení KS:

- SW řešení
 - použití algoritmu pro vstupní a výstupní sekci
- HW řešení
 - využití speciálních instrukcí procesoru
- řešení OS
 - nabízí programátorovi prostředky pro řešení KS (datové typy a funkce)
- řešení programovacího jazyka
 - konkurenční programování

Předpoklady pro řešení KS

- procesy se provádějí nenulovou rychlostí
- žádné předpoklady o relativní rychlosti procesů
- uvažujeme i víceprocesorové systémy
 - předpoklad: paměťové místo smí v jednom okamžiku zpřístupnit vždy pouze jediný procesor
 - nelze předpokládat, že se budou procesy pravidelně střídat v běhu
- stačí specifikovat vstupní a výstupní sekci

Požadované vlastnosti řešení KS

- vzájemné vylučování = mutual exclusion
 - zajištění exkluzivního přístupu (vždy pouze jeden proces v KS)
- pokrok v přidělování
 - rozhodování musí proběhnout v konečném čase, přičemž do rozhodování nesmějí zasahovat procesy provádějící svou zbytkovou sekci
 - žádný proces nesmí čekat do nekonečna
- omezené čekání
 - pouze omezený počet jiných procesů smí získat přístup do kritické sekce před vyhověním žádosti o vstup danému procesu

Příklady nevhodných SW řešení

Algoritmus 1

```
locked = 0;           // sdílená proměnná evidující obsazenost KS

// kód procesu
repeat
    while (locked != 0);    // čekání, dokud není KS volná
    locked = 1;             // nastavení obsazené KS
    // kód KS
    locked = 0;             // uvolnění KS
    // kód ZS
forever
```

Požadavek vzájemného vylučování není splněn! (můžou přijít oba najednou).

- Důvod: při přepnutí kontextu oba procesy postupně nastaví, že jsou v KS

Algoritmus 2

```
turn = 0; // sdílená proměnná určující proces s právem vstupu do KS

// Kód procesů Pi
repeat
    while (turn != i);    // čekání, dokud není Pi na řadě
    // kód KS
    turn = 1 - i;         // KS je volná pro jiný proces
    // kód ZS
forever
```

Požadavek pokroku není splněn! (i když požadavek vzájemného vylučování ano)

- Důvod: pokud P₀ již nepožaduje vstup do KS a P₁ jej opětovně požaduje, bude P₁ trvale čekat, protože P₀ už nikdy nezvýší hodnotu proměnné `turn`. Vyhladovění procesu P₁!

Algoritmus 3

```
flag[i] = false;          // sdílené proměnné určující požadavek vstupu do
                           // KS procesem Pi
// Kód procesů Pi:
repeat
    flag[i] = true;        // nastavení požadavku vstupu do KS
                           // procesem Pi
    while (flag[1 - i]);   // čekání, dokud je druhý proces v KS
    // kód KS
    flag[i] = false;       // KS není požadována
    // kód ZS
forever
```

Pokud se oba procesy rozhodnout vstoupit do KS ve stejnou dobu, budou oba čekat na KS aktivním čekáním (while) = deadlock!

Petersonův algoritmus

!! Petersonův algoritmus funguje pro 2 procesy, pro víc ne !!

```
flag[i]:
flag[0] = false;          // sdílené proměnné určující požadavek vstupu do
flag[1] = false;          // KS procesy P0 a P1

turn = 0;                  // sdílená proměnná určující proces s právem
                           // přednosti vstupu do KS

// Kód procesů Pi:
repeat
    flag[i] = true;        // nastavení požadavku vstupu do KS
                           // procesem Pi
    turn = 1 - i;          // přednost má druhý proces
    while (flag[1 - i]     // čekání, dokud druhý proces požaduje KS
           && turn != i);  // a současně má druhý proces přednost
    // kód KS
    flag[i] = false;       // KS není požadována
    // kód ZS
forever
```